degree. The Gaussian deformation field $\{\mathcal{E}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}\}$ consists of an encoder part $\mathcal{E}(\mathcal{G}_{\text{canon}}, t)$, which outputs an embedding voxel feature f_d based on spatio-temporal input coordinates p and t, and a decoder part $\mathcal{D}$, which decodes the voxel feature into each Gaussian's deformation $\Delta\mathcal{G}_t$. Note that, to ensure Gaussian deformations resemble real-world physical motion, 4DGS computes deformation values only for Gaussian position p, scale s, and rotation r. Therefore, $\Delta\mathcal{G}_t$ can be expressed as $\{\Delta p_{t,i}, \Delta s_{t,i}, \Delta r_{t,i}\}_{i=1}^{N}$. By repeatedly adding the outputs of the deformation field $\Delta\mathcal{G}_t = \mathcal{D}(\mathcal{E}(\mathcal{G}_{\text{canon}}, t))$ to the canonical 3D Gaussians $\mathcal{G}_{\text{canon}}$ at each timestep t, we can render an image $\hat{I}_{M,t}$ from deformed 3D Gaussians $\mathcal{G}_{\text{def},t} = \mathcal{G}_{\text{canon}} + \Delta\mathcal{G}_t$ as: $\hat{I}_{M,t} = S(M, \mathcal{G}_{\text{def},t})$, where M denotes the camera matrix, and S represents the rendering (differential splatting) process of the 3DGS.

Encoder for Gaussian Deformation Field. 4DGS incorporates Hexplane [5, 15], as a core component in the structure of the encoder $\mathcal{E}(\mathcal{G}_{\text{canon}}, t)$ within its Gaussian deformation field. The Hexplane is a spatio-temporal structure encoder and can be viewed as a generalization of Triplane [6], which was originally designed to embed spatial information in 3D space.

Hexplane-based encoder $\mathcal{E}(\mathcal{G}_{\text{canon}}, t)$ can be parametrized by six multi-resolution voxel grids R_l across the four dimensions (x, y, z, t) and simple MLP encoder ϕ_d as $\mathcal{E}(\mathcal{G}_{\text{canon}}, t) = \{R_l(i,j), \phi_d | (i,j) \in \{(x,y),(y,z),(x,z),(x,t),(y,t),(z,t)\}, l \in \{1,2\}\}$, where l represents the multi-resolution level (the multi-resolution technique is relevant to Instant-NGP [39], enabling fast optimization and rendering).

The spatio-temporal embedding voxel feature f_d is obtained from the Hexplane as $f_d = \phi_d(f_h)$, where $f_h = \bigcup_l \prod \text{interp}(R_l(i,j))$, and $(i,j) \in \{(x,y),(x,z),(y,z),(x,t),(y,t),(z,t)\}$. In 4DGS, the x, y, and z coordinates of the Gaussian position p and timestep t is used to query voxel features across six planes. The six voxel features obtained from each plane through bilinear interpolation are then combined via the Hadamard product (channel-wise product). This queried voxel feature f_h is subsequently passed through ϕ_d to yield the final embedding voxel feature f_d as shown in Fig. 2.

Decoder for Gaussian Deformation Field. The spatio-temporal voxel feature f_d passes through a multi-head simple MLP decoder $\mathcal{D} = \{\phi_p, \phi_s, \phi_r\}$, which decodes it into the deformation values of the Gaussian feature p, s and r as $\Delta p = \phi_p(f_d), \Delta s = \phi_s(f_d)$, and $\Delta r = \phi_r(f_d)$.

Since the deformation field $\{\mathcal{E}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}\}$ is designed with a compact Hexplane and a simple MLP, 4DGS achieves real-time rendering speed. This provides a significant advantage for editing tasks, where rendering is performed frequently. To leverage this efficiency and the static-dynamic separability, we applied 4DGS to our primary representation for 4D dynamic scenes.

4. Method

In this section, we present **Instruct-4DGS**, our proposed method for efficient dynamic scene editing, as illustrated in Fig. 3. (Sec. 4.1) We first train dynamic scenes for editing targets by optimizing the 4D Gaussian Splatting (4DGS) [63]. (Sec. 4.2) Motivated by the static-dynamic separability of Hexplane [5, 15]-based 4DGS, we initially focus on editing the static canonical 3D Gaussians [24] to efficiently edit the dynamic scene. (Sec. 4.3) To mitigate overfitting issues that may arise during the 3D Gaussian editing process and refine the motion artifacts in the dynamic scene, we introduce a temporal refinement stage using score distillation.

4.1. Optimizing 4D Gaussians for Target Scenes

Our dynamic scene editing requires a 4D Gaussian representation of the target dynamic scene. To obtain this, we optimize the 4D Gaussians and use it for editing. Specifically, we use dynamic scene datasets [28, 50] composed of multi-camera captured videos, which can be represented as a set of images $\{I_{M,t}\}$ in which M denotes the camera matrix and t denotes the timestep within the videos. We synthesize images $\hat{I}_{M,t}$ by rendering the randomly initialized dynamic scene $\{\mathcal{G}^{\text{init}}_{\text{canon}}, \mathcal{E}^{\text{init}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{init}}\}$. Then we calculate the RGB L1 loss against the corresponding dataset image $I_{M,t}$, training the dynamic scene through this process. We also apply a grid-based total variational loss [5, 14, 15, 56], $\mathcal{L}_{\text{TV}}$ to enforce smoothness in the deformation field output along the timesteps. Note that, as our editing method is highly dependent on the quality of the target dynamic scene, incorporating such regularization loss is helpful. The entire loss function used for 4DGS training is: $\mathcal{L}_{\text{4DGS}} = |\hat{I}_{M,t} - I_{M,t}| + \mathcal{L}_{\text{TV}}$.

As a result, we obtain the optimized 4D Gaussians $\left\{\mathcal{G}^{\text{opt}}_{\text{canon}}, \mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\right\}$, which represent the editing target scene. The static component $\mathcal{G}^{\text{opt}}_{\text{canon}}$ serves as the main editing target in Sec. 4.2–4.3. In Sec. 4.2, we edit the static component by modifying only the images corresponding to the first timestep, ensuring efficient editing. In Sec. 4.3, we refine the edited static component $\mathcal{G}^{\text{edit}}_{\text{canon}}$ to better align with the original deformation field $\{\mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\}$ using score-based temporal refinement, mitigating potential motion artifacts. A more detailed training setup follows [63].

4.2. Stage 1: Efficient Dynamic Scene Editing with Static 3D Gaussians

In Sec. 4.1, we obtained the optimized canonical 3D Gaussians $\mathcal{G}^{\text{opt}}_{\text{canon}}$, which models the explicit appearance and geometry of a 3D scene, along with the optimized dynamic components $\mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t)$ and $\mathcal{D}^{\text{opt}}$. For efficient dynamic

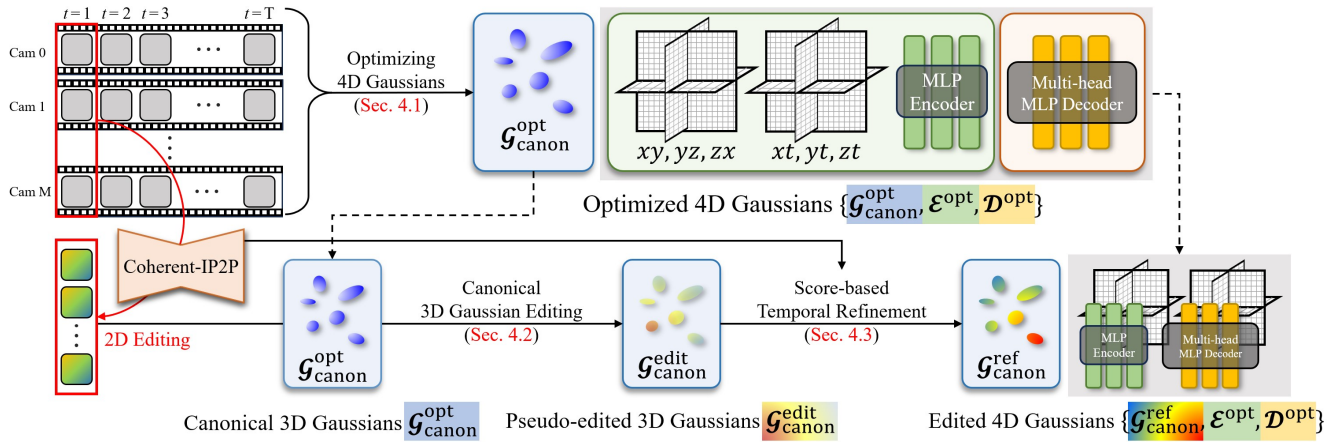


Figure 3. **Overall pipeline of our proposed dynamic scene editing method (Instruct-4DGS)**: To obtain the target dynamic scene for editing, we first optimize the 4D Gaussians using a multi-camera captured video dataset (Sec. 4.1). We then perform 3D Gaussian editing on the static canonical 3D Gaussians by editing only the multiview images corresponding to the first timestep (Sec. 4.2). We apply score-based temporal refinement to mitigate motion artifacts without additional image editing (Sec. 4.3).

scene editing, we perform editing only on $\mathcal{G}^{\text{opt}}_{\text{canon}}$, which is minimal but sufficient information for visual editing of the dynamic scene as shown in Fig. 3.

To generate supervision images for editing the $\mathcal{G}^{\text{opt}}_{\text{canon}}$, we extract a subset of multiview images fixed at the initial timestep and then edit them using InstructPix2Pix [4]. Subsequently, we edit optimized canonical 3D Gaussians $\mathcal{G}^{\text{opt}}_{\text{canon}}$ with an L1 RGB loss supervising the edited images. Compared to the latest 4D editing method Instruct 4D-to-4D [38], which requires editing $T \times \mathcal{M}$ images—where T is the number of video timesteps and $\mathcal{M}$ is the number of cameras—through iterative dataset updates, our approach significantly reduces the computation required to address the editing of the dynamic scene. Moreover, this approach allows rapid transitions to the edited result, regardless of the number of timesteps T of the dynamic scene. After completing the 3D Gaussian editing process, we obtain a ***pseudo-edited dynamic scene*** $\{\mathcal{G}^{\text{edit}}_{\text{canon}}, \mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\}$, which is obtained by simply recombining the edited canonical 3D Gaussians $\mathcal{G}^{\text{edit}}_{\text{canon}}$ with the original Gaussian deformation field $\{\mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\}$.

To ensure spatial consistency of the 3D Gaussian editing process, we utilize Coherent-IP2P [38, 53, 64], which replaces the 2D convolutional layer (self-attention module) with a 3D convolutional layer (cross-attention module), similar to Instruct 4D-to-4D (by reusing the original parameters of kernels). As shown in Fig. 8, this encourages collaborative editing among images within the multiview subset, preventing the results from becoming blurry. The entire editing process for the static canonical 3D Gaussian editing can be completed within a few tens of minutes by editing only multiview images of a single timestep and performing a few hundred 3DGS editing iterations.

4.3. Stage 2: Refinement using Score Distillation for Temporal Alignment

After the first editing stage proposed in Sec. 4.2, the *pseudo-edited dynamic scene* $\{\mathcal{G}^{\text{edit}}_{\text{canon}}, \mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\}$ exhibits severe motion artifacts, as shown in Fig. 4 (a). The primary cause is the slight shift in the positions p of Gaussian primitives in $\mathcal{G}^{\text{opt}}_{\text{canon}}$ during the 3D Gaussian editing process, which results in discrepancies between the queried embedding voxel feature f_h and those of the original dynamic scene. Moreover, only the Spherical Harmonics (SH) colors on the surface visible at the initial timestep are updated. As a result, if the Gaussian primitives in *pseudo-edited* $\mathcal{G}^{\text{edit}}_{\text{canon}}$ rotate at later timesteps, unedited SH values that were previously hidden may become exposed, leading to artifacts. Therefore, we introduce a temporal refinement stage to resolve the misalignment between the original deformation field $\{\mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\}$ and the edited canonical 3D Gaussians $\mathcal{G}^{\text{edit}}_{\text{canon}}$.

To perform the refinement stage efficiently without editing additional dataset images, we employ the score distillation mechanism [43]. Since the dynamic scene is edited using multiple 2D images generated by IP2P, the prior of the 2D diffusion model (*i.e.*, IP2P) can be distilled into the 4D dynamic scene. The editing process can be continued using the noise prediction loss (*i.e.* score) obtained from each IP2P inference as Eqs. 1 and 2. Since we just use score distillation for editing refinement rather than generation or editing from scratch, this stage can be completed with a smaller number of iterations. Consequently, our approach is relatively less affected by inherent issues of Score Distillation Sampling (SDS), such as *Janus problem* [53].

Similar to Sec. 4.2, we apply Coherent-IP2P with the diffusion prior θ and observe that it reduces blur-

ring effects and enhances qualitative performance compared to the original IP2P. At each refinement iteration, we rendered B images of the *pseudo-edited dynamic scene* $\tilde{I} = \left\{\hat{I}_i = S(M_i, \mathcal{G}^{\text{edit}}_{\text{def},t_i})\right\}_{i=1}^{B}$ using random camera matrices $\{M_i\}_{i=1}^{B}$ and random timesteps $\{t_i\}_{i=1}^{B}$ as input for Coherent-IP2P, where S denotes the rendering process of the 3DGS (subscripts M and t on $\hat{I}$ omitted for simplicity). We optimize $\mathcal{G}^{\text{edit}}_{\text{canon}}$ using the following SDS loss to obtain the refined 3D Gaussians $\mathcal{G}^{\text{ref}}_{\text{canon}}$:

$$\nabla_{\mathcal{G}^{\text{edit}}_{\text{canon}}} \mathcal{L}_{\text{SDS}} = \mathbb{E}_{t,\tilde{t},\epsilon,\mathcal{M}} \left[\left(\epsilon_\theta \left(\tilde{I}, c_I, c_T; t, \tilde{t}, \mathcal{M} \right) - \epsilon \right) \frac{\partial \tilde{I}}{\partial \mathcal{G}^{\text{edit}}_{\text{canon}}} \right], \tag{1}$$

$$\begin{aligned} \epsilon_\theta(\tilde{I}, c_I, c_T) = \epsilon_\theta(\tilde{I}, \emptyset, \emptyset) + s_I \left(\epsilon_\theta(\tilde{I}, c_I, \emptyset) - \epsilon_\theta(\tilde{I}, \emptyset, \emptyset) \right) \\ + s_T \left(\epsilon_\theta(\tilde{I}, c_I, c_T) - \epsilon_\theta(\tilde{I}, c_I, \emptyset) \right) \end{aligned} \tag{2}$$

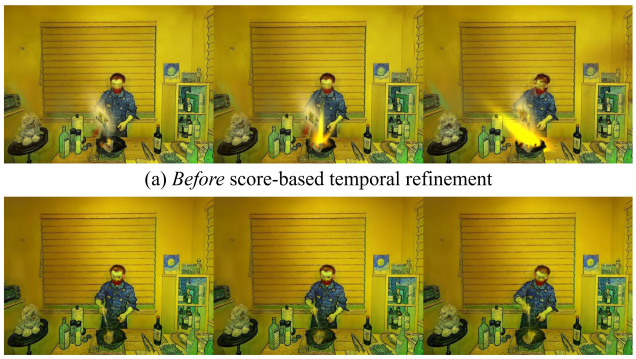
, where $\tilde{t}$ is diffusion timestep, ϵ is diffusion noise, $c_I = \{I_i\}_{i=1}^{B}$ is original dataset images, c_T is user instruction, s_I and s_T are Classifier-Free-Guidance [20] scale for c_I and c_T, $\epsilon_\theta(\tilde{I}, c_I, c_T)$ is Coherent-IP2P denoiser networks including VAE [26]. This score-based guidance encourages a set of rendered 2D images from the *pseudo-edited* 4D Gaussians at arbitrary timesteps $\tilde{I}$ to resemble the edited images that IP2P would generate based on the c_I and c_T, thereby effectively refining motion artifacts. As a result, we obtain refined canonical 3D Gaussians $\mathcal{G}^{\text{ref}}_{\text{canon}}$ that aligns well with the original deformation field $\left\{\mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\right\}$ while maintaining the edited appearance. After the refinement stage, we obtain a completely edited 4D dynamic scene which is represented as $\left\{\mathcal{G}^{\text{ref}}_{\text{canon}}, \mathcal{E}^{\text{opt}}(\mathcal{G}_{\text{canon}}, t), \mathcal{D}^{\text{opt}}\right\}$.

5. Experiments

5.1. Experimental Setup

Datasets. We use DyNeRF [28] and Technicolor [50], a real-world multiview video dataset, to train and edit 4D dynamic scenes. The DyNeRF dataset includes six 10-second video sequences captured at 30 fps by 15 to 20 cameras with a face-forward perspective. Technicolor includes a wider variety of motion and scenarios, captured with 16 cameras. For comparison with the baseline [4], we trim the videos into 50-frame-long segments. We have also included the results on monocular datasets [16, 41] in the supplementary.

Baselines. We conduct a qualitative and quantitative comparison with Instruct 4D-to-4D [38], the only prior work addressing instruction-guided 4D dynamic scene editing. Instruct 4D-to-4D utilizes NeRFPlayer [55] as its backbone 4D representation and employs an iterative dataset update method, which involves editing all 2D images used for synthesizing the dynamic scene. It utilizes optical flow-based warping [58] and depth-based warping to ensure consistency across all edited 2D images. To alleviate the time-consuming dataset update process, Instruct 4D-to-4D employs two GPUs in parallel: one for the dataset update thread and the other for the dynamic scene editing thread.



(a) *Before* score-based temporal refinement

(b) *After* score-based temporal refinement

Figure 4. **Effectiveness of score-based temporal refinement**: Score-based temporal refinement effectively resolves misalignment between the canonical 3D Gaussians and the original deformation field that arises during the 3D Gaussian editing process. Without requiring additional 2D image updates, this process completes dynamic scene editing within a few hundred iterations.

Implementation Details. In Sec. 4.1, we follow the experimental settings of [63]. Throughout the experiments utilizing InstructPix2Pix [4], we set the CFG [20] scales for image condition and text instruction to 1.2 and 8.5 to 10.5, respectively. In the 3D Gaussian editing stage (Sec. 4.2), we train for 800 to 1000 iterations, depending on the editing style. For the score-based refinement stage (Sec. 4.3), an average of 800 iterations is sufficient to complete the dynamic scene editing successfully. All experiments are conducted using a single NVIDIA A40 GPU.

5.2. Results

Quantitative Results. To quantitatively evaluate the visual quality of the edited dynamic scene, we measure PSNR, SSIM [61], and LPIPS [69] between the 2D multi-view images used as supervision for dynamic scene editing and the images rendered from the edited dynamic scene using the corresponding camera parameters. Additionally, to assess how well the edited dynamic scene aligns with the input instruction, we also measure CLIP [45] similarity.

Table 1 presents a quantitative comparison of our method, Instruct-4DGS, and the baseline, Instruct 4D-to-4D, on DyNeRF. While our method shows slightly worse PSNR and SSIM in some cases, this is expected due to our efficient editing strategy. Unlike the baseline, which directly optimizes pixel-level accuracy using all edited images as training targets, our approach optimizes the dynamic scene using only instructions, without additional image editing during the temporal refinement stage. Consequently, pixel-wise accuracy (*i.e.*, PSNR and SSIM) may be worse, but our method demonstrates superior perceptual quality, as shown in the consistently lower LPIPS across all cases. Additionally, our approach excels in instruction-